Python Programming Fundamentals

Protocols and Structural Typing

Programming Fundamentals Team

SETU

2026-06-12

Outline

Outline

1.1 Nominal vs Structural

Nominal typing (ABC)	Structural typing (Protocol)
Must explicitly inherit from ABC	No inheritance needed
`isinstance()` checks class hierarchy	`isinstance()` checks structure (if `@runtime_checkable`)
Formal, rigid	Flexible, duck-typing friendly

Outline

2. Defining a Protocol

```python
from typing import Protocol

class Drawable(Protocol):
    def draw(self) -> str:
        ...   # body is "..." — not implemented

class Circle:
    def draw(self) -> str:
        return "Drawing circle ○"

class Square:
    def draw(self) -> str:
        return "Drawing square □"

# Neither inherits from Drawable!
def render(item: Drawable) -> None:
```

2. Defining a Protocol

```
    print(item.draw())

render(Circle())    # works ✓
render(Square())    # works ✓
```

Outline

3. Protocol vs ABC

```python
# ABC approach — must inherit
from abc import ABC, abstractmethod

class Drawable(ABC):
    @abstractmethod
    def draw(self): pass

class Circle(Drawable):   # must say "(Drawable)"
    def draw(self): return "○"


# Protocol approach — no inheritance needed
from typing import Protocol

class Drawable(Protocol):
    def draw(self) -> str: ...
```

3. Protocol vs ABC

```
class Circle:                # no mention of Drawable
    def draw(self): return "○"
```

Both work for type checking — Protocols are more flexible.

Outline

4. @runtime_checkable

```
from typing import Protocol, runtime_checkable

@runtime_checkable
class Drawable(Protocol):
    def draw(self) -> str: ...

class Circle:
    def draw(self) -> str: return "○"

class Triangle:
    pass   # no draw()

c = Circle()
t = Triangle()
```

4. @runtime_checkable

```
isinstance(c, Drawable)   # True  ← has draw()
isinstance(t, Drawable)   # False ← missing draw()
```

Without `@runtime_checkable`, `isinstance` raises `TypeError`.

Outline

5. Practical Protocol Example

```python
from typing import Protocol

class Serialisable(Protocol):
    def to_dict(self) -> dict: ...
    def from_dict(cls, d: dict) -> "Serialisable": ...

# Any class with these methods is automatically Serialisable
# — Product already qualifies if it has to_dict() and from_dict()!

def save_all(items: list[Serialisable], filename: str):
    import json
    data = [item.to_dict() for item in items]
    with open(filename, "w") as f:
        json.dump(data, f, indent=2)
```

Outline

6. When to Use ABC vs Protocol

Use ABC when…	Use Protocol when…
You control all the classes	You don’t control all classes
You want to prevent instantiation	You want flexible matching
You have shared concrete methods	Only an interface matters
Hierarchy is important	Behaviour is what matters

Outline

7. Summary

- **ABC** — formal contract, explicit inheritance, cannot instantiate
- **Protocol** — structural contract, no inheritance needed, duck-typing style
- **`@abstractmethod`** — forces subclass to implement
- **`@runtime_checkable`** — allows `isinstance()` with Protocol

Thanks for Watching - Any questions?